



MYTHIC

Analog Compute Engine (ACE) Operation

June 2026

Document Version v0.2
SDK Version 26.05

1	Introduction	3
2	ACE Architecture	3
2.1	Circuits For Matrix Multiplication.....	3
2.1.1	AIDAC.....	4
2.1.2	NVM Cell.....	5
	NVM Cell Fundamentals	5
	NVM Cells as Weight Storage / Usage	5
	Programming.....	6
	Weight	7
2.1.3	ADC	8
2.2	ACE Array Structure	9
2.2.1	Floorplan.....	10
2.2.2	Tile Configuration and Area Estimate.....	10
3	ACE Input and Output Interface	11
3.1	ACE Timing	12
3.1.1	ACE Timing Diagram.....	12
4	Electrical Specifications	12
5	ACE Modeling	13
5.1	Accuracy Simulator vs. SystemVerilog / RNM	13
5.1.1	Hierarchy of Modeling Methods.....	13
5.1.2	Mythic Modeling Process	14
5.2	Error Sources In Modeling	16
5.2.1	What Is and Is Not Being Modeled.....	16
	Examples of Errors Being Modeled.....	16
	Examples of Errors NOT Being Modeled	16
5.2.2	ACE Signal Chain With Error Sources	16
6	ACE Design For Manufacturing.....	17
6.1	Built-In-Self-Test.....	17
6.2	Circuit Monitoring and Temperature Compensation	17
6.3	NVM Cell Failures, Testing and Yield Enhancement:.....	18
6.4	Redundancy	20

1 Introduction

The purpose of the ACE is to perform a matrix multiplication used in AI inference computation. While the input vectors, weight vectors, and output vectors are all digital, the multiplication is performed in the analog domain which allows the ACE to outperform traditional digital architectures.

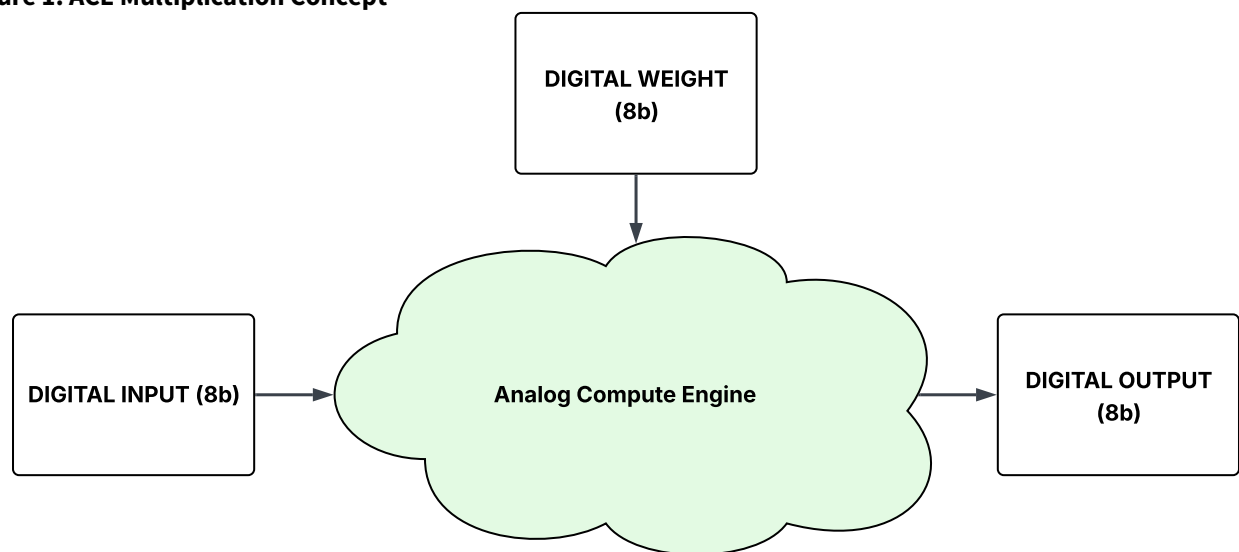
Matrix multiplication:

We don't have a way to export this macro.

Terms re-named for simplicity:

We don't have a way to export this macro.

Figure 1. ACE Multiplication Concept



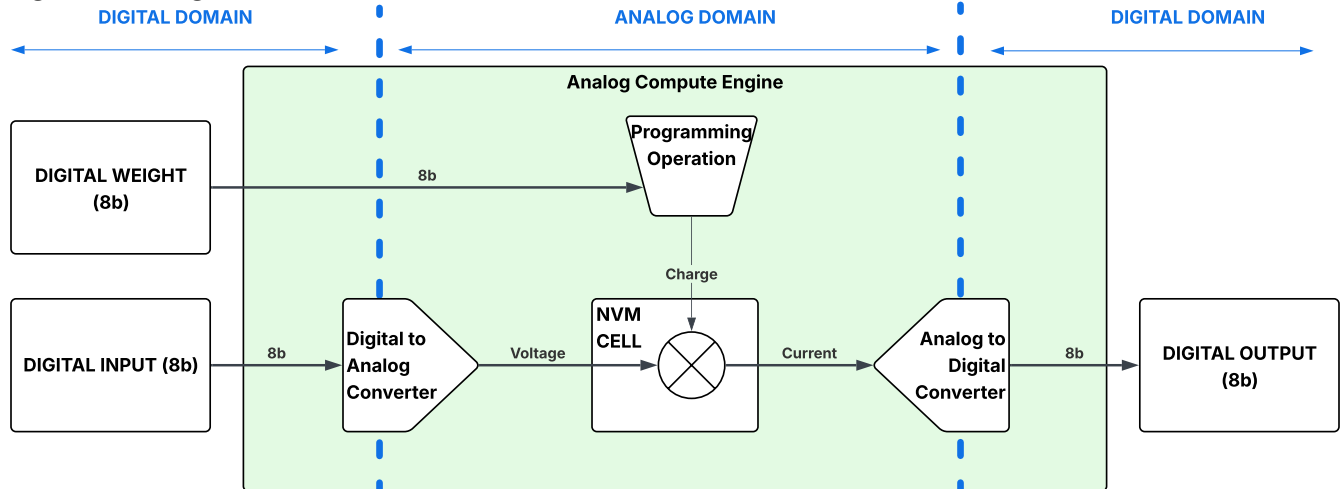
Keeping the inputs and outputs of the ACE in the digital domain allows for complete flexibility of data flow, data processing, and robust data transmission outside of the ACE, and containing all concerns about analog domain processing as small as possible.

2 ACE Architecture

2.1 Circuits For Matrix Multiplication

The three most critical analog signal chain blocks are the Digital-to-Analog Converter (AIDAC), NVM Cell, and Analog-to-Digital Converter (ADC):

Figure 2. ACE Signal Chain Concept

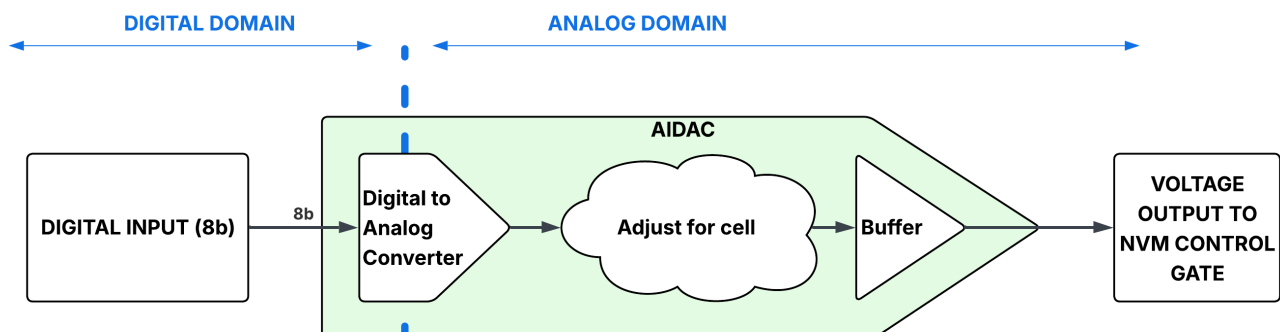


- AIDAC
 - (AI) Digital-to-Analog Converter
 - Proprietary technology of Mythic
 - Converts the digital input value into an analog voltage
- NVM FLASH cell
 - Proprietary technology of Microchip (SST) and Mythic
 - Stores the digital weight value as an analog charge used as a threshold voltage
 - Outputs a current which is a multiplication of input voltage and weight value
- ADC
 - Analog-to-Digital Converter
 - Proprietary technology of Mythic
 - Converts an input current into a digital output value representing the sum of input and weight multiplication products

2.1.1 AIDAC

The AIDAC converts an 8b digital input to a corresponding/proportional analog voltage that is sent to the control gates of the NVM cells.

Figure 3. AIDAC Diagram



The processing flow for the AIDAC is:

- A relatively conventional Digital-to-Analog Converter converts the 8b digital input to an analog signal
- Mythic performs proprietary signal processing that ensures the analog signal is matched to the correct conditions for the NVM cell
- The analog signal is then buffered before being sent to the row of NVM cells

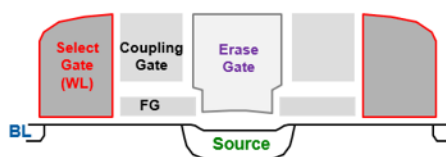
2.1.2 NVM Cell

NVM Cell Fundamentals

Figure 4. Mythic 28nm GEN2 Flash Technology

Mythic 28nm GEN2 Flash Technology

(Licensed from SST SuperFlash ESF3-28)



Mythic GEN2 Flash Cells		Low power high performance Flash cells
Proven for decades	⇒	Floating Gate/Production in Billions of chips
CMOS process	⇒	NMOS based
Fine grid low power erase	⇒	Selected tunneling erase (2 rows)
Low power programming	⇒	Source-side Injection (<1uA/cell)
Low RC array	⇒	Self-aligned process with strap cells
Excellent reliability	⇒	Retention high (likely no refresh)
Compact	⇒	Small Cell size < 0.2u ²
Excellent endurance	⇒	Endurance very high

NVM Cells as Weight Storage / Usage

The NVM cell is the heart of the Analog Compute Engine, and performs two critical tasks:

- The differential NVM cell stores the matrix weights as a charge (during programming)
- The differential NVM cell performs the input and weight multiplication

The NVM differential cell current is a function of both the programmed weight and the input voltage on its control gate:

 Broken macro where  Broken macro and  Broken macro is the control gate voltage which is the input from the AIDAC.

The cell currents are summed on the bitline which is routed to an ADC.

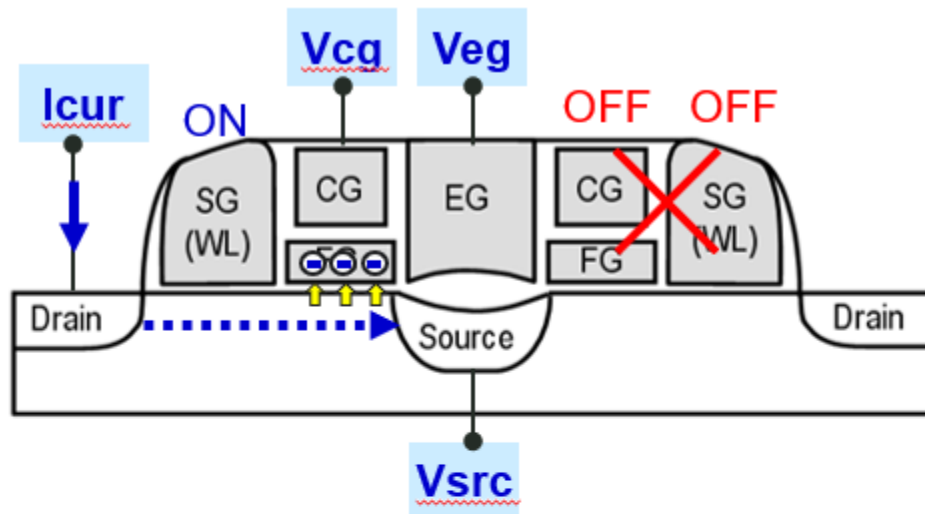
Programming

To program,

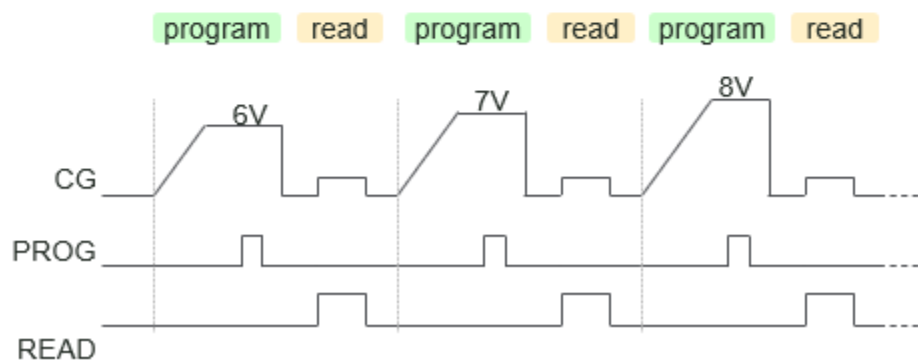
- High voltage is generated from a charge pump and applied to the control gate (CG)
- A current is injected into the side of the cell and get trapped into the floating gate, effectively changing the V_t of the cell.
- A sequence of programming and read (verify) pulses is required to accurately move the V_t of the cell (shown in Example A)

MYTHIC

Figure 5. GEN2 Cross Section and Basic Programming Sequences



Example A

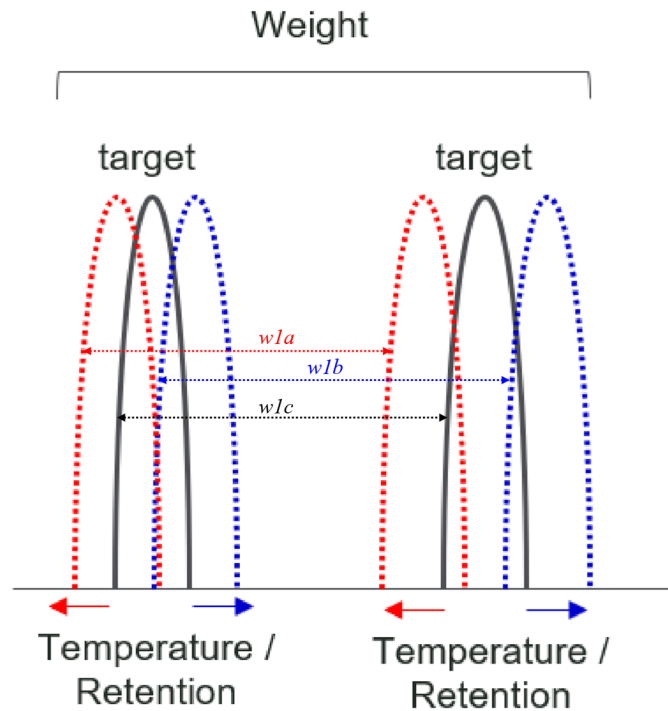


Weight

- Weight by definition is a pair of different cells.
- Weight noises can be affected by cell fundamentals:
 - Cell leakage (weight decay)
 - Weight decay is a leakage of charge from the flash cell, which causes the weight magnitude to decrease over time.
 - For our cells, charge retention is high (projected 10 years) with minimal leakage.
 - Temperature
 - Weight can drift due to temperature changes
 - For our cells, we have capability to compensate during inference by adjust CG/EG level

- Behaviors are known/trained at system level
- Since the weight is the current differential from two current component ($w1a$, $w1b$, $w1c$) having a similar effect, the actual weight is much less than the absolute current change due to temperature or retention (mostly the same direction)

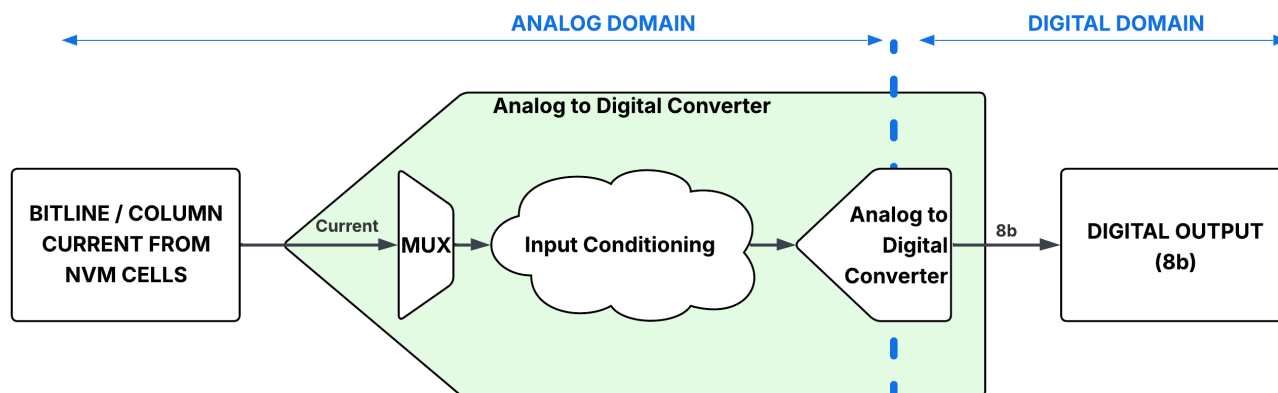
Figure 6. NVM Weight - Differential Cell



2.1.3 ADC

The ADC measures the bitline current from the NVM cells, and converts it to an 8b digital value.

Figure 7. ADC Diagram



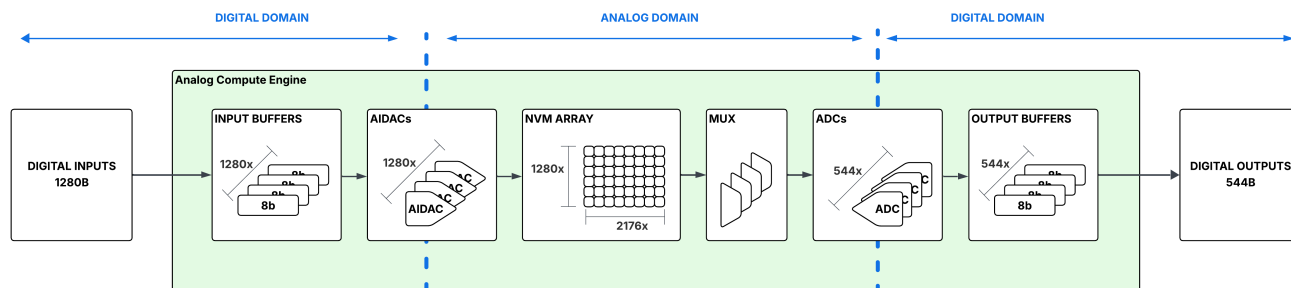
The processing flow for the ADC is:

- An input mux selects a differential pair of bitline/column currents
- Mythic performs proprietary signal processing that preserves the analog signal while optimizing the input to the ADC
- The analog signal is processed by a relatively conventional Analog-to-Digital Converter into an 8b output word that represents the magnitude of the differential current input

2.2 ACE Array Structure

The full ACE size is 1280 input rows by 1,088 differential output columns.

Figure 8. ACE Signal Chain More Detail



The list of mixed-signal components involved in a matrix multiply operation is:

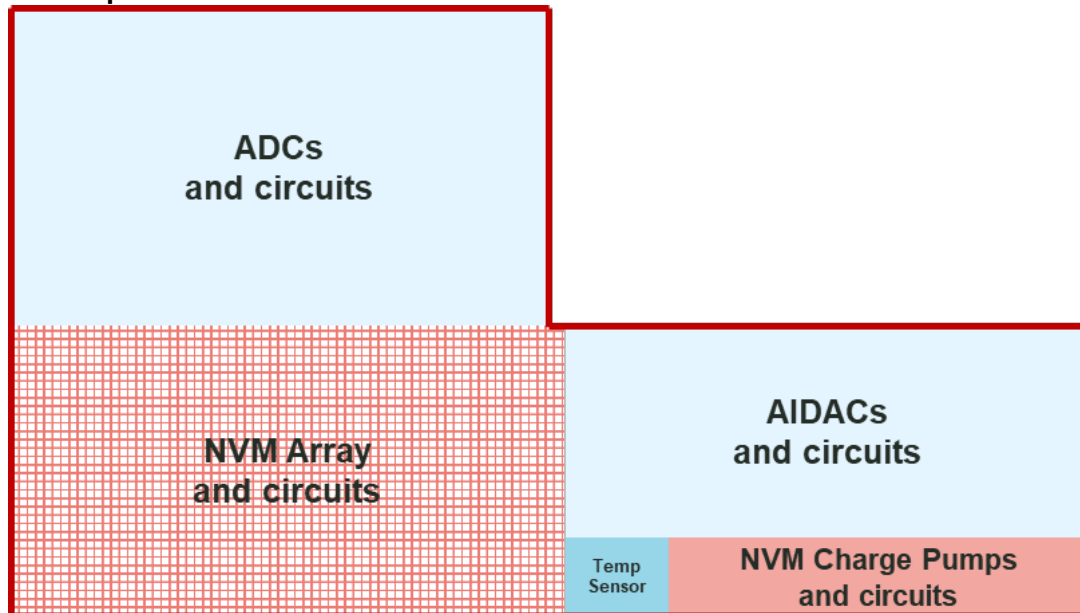
- Input Buffers - these sort the 1280 Bytes of digital input data into the 1280 8-bit rows of the of AIDACs
- 1280 AIDACs - these convert an 8b digital input to an analog voltage to the NVM cell
- NVM Array - 1280 rows * 2176 columns (1088 differential pairs) = 2,785,280 cells
- 544 ADCs with 2:1 or 4:1 input mux - these convert the bitline current representing the matrix multiplication to an 8b digital output value
- 544 Output buffers collect and shift the ADC data to the digital processing at the layer above

2.2.1 Floorplan

A single ACE consists of:

- NVM Array and support circuits
- NVM Charge pump and support circuits
- AIDACs and supporting circuits
- ADCs and support circuits
- Temperature Sensor

Figure 9. ACE Floorplan

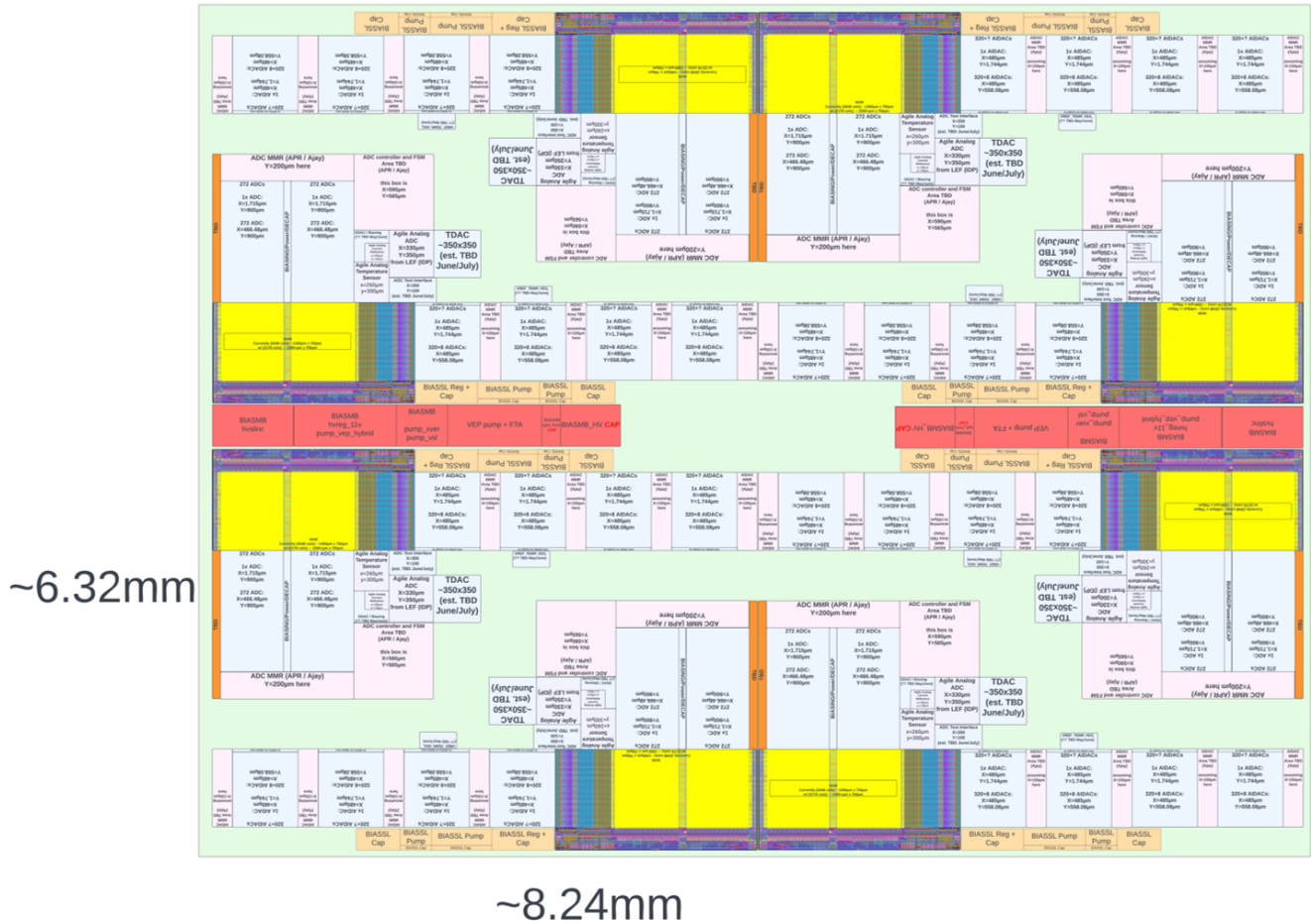


2.2.2 Tile Configuration and Area Estimate

A single ACE tile will consist of 4 ACEs. Each tile will be a standalone unit with dedicated I/O and power, which can be tiled throughout the analog wafer to increase or decrease the total amount of compute.

The floorplan below shows 2x ACE tiles together; there are a total of 8 ACEs). The current area estimate of 2 neighboring ACE tiles is represented with pre-shrink numbers, at roughly 8.24mm x 6.32mm. **One ACE tile is half of this height, with X=8.24mm and Y=3.16mm**

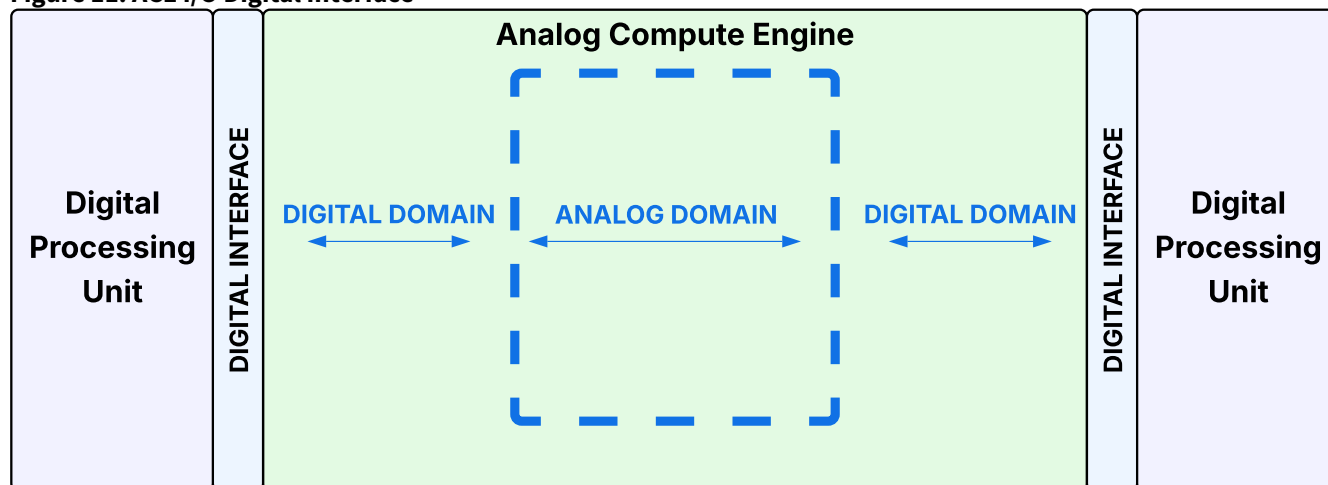
Figure 10. 2x ACE Tile Layout Floorplan



3 ACE Input and Output Interface

The ACE uses a fully digital interface to load the input buffer and unload the output buffer to the Digital Processing Unit above.

Figure 11. ACE I/O Digital Interface

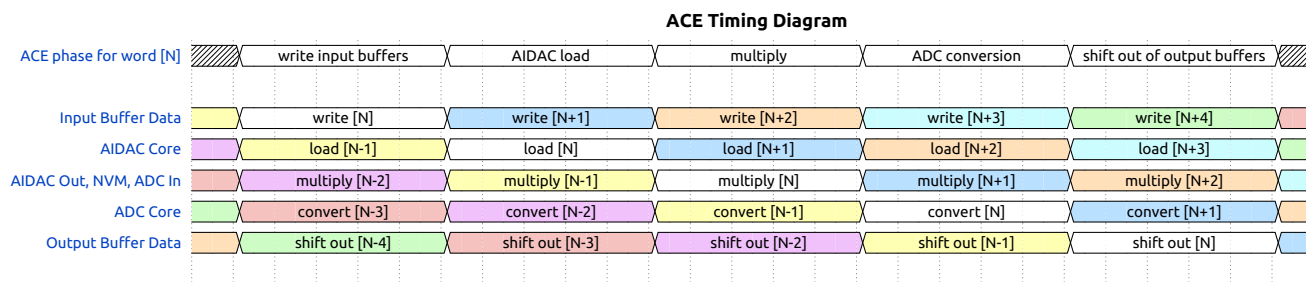


3.1 ACE Timing

The ACE performs one matrix multiply every **160ns**; this means the **throughput rate** for the ACE is 160ns. Due to the use of pipelining, the processing **latency** for each input to the ACE is **5 cycles**.

3.1.1 ACE Timing Diagram

Figure 12. ACE Timing Diagram



4 Electrical Specifications

Note: more details will be provided as package-level information becomes available.

Pin	Description	Min	Typ	Max	Unit
VDDA	Core analog / mixed-signal supply	0.95	1	1.05	V
VDD	Core supply	0.95	1	1.05	V

Pin	Description	Min	Typ	Max	Unit
VDDA18	Higher voltage analog / mixed-signal supply	1.71	1.8	1.89	V
VDD18	Higher voltage digital and IO device supply	1.71	1.8	1.89	V
VSSA	Analog ground		0		V
VSS	Ground		0		V

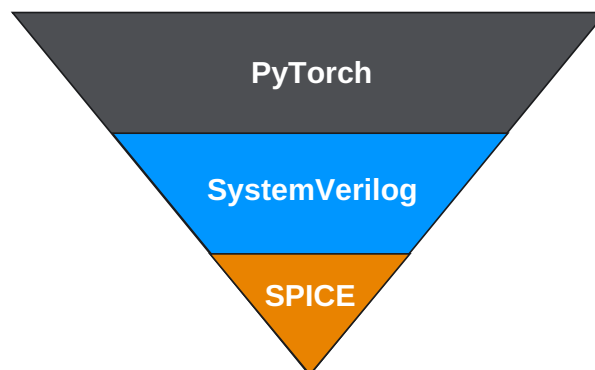
5 ACE Modeling

5.1 Accuracy Simulator vs. SystemVerilog / RNM

5.1.1 Hierarchy of Modeling Methods

Mythic is using a hierarchy of methods to verify system and network accuracy:

Figure 13. Modeling Hierarchy



1. SPICE simulation

- SPICE is the most accurate circuit simulation method, but it is the most computationally expensive
- We simulate our ADCs, AIDACs, NVM, and other analog circuits with SPICE to characterize our circuit behavior and create **reference models**
- Our reference model behavior matches SPICE simulations
- SPICE is capable of simulating thousands of devices, but not running a full computation of the entire ACE

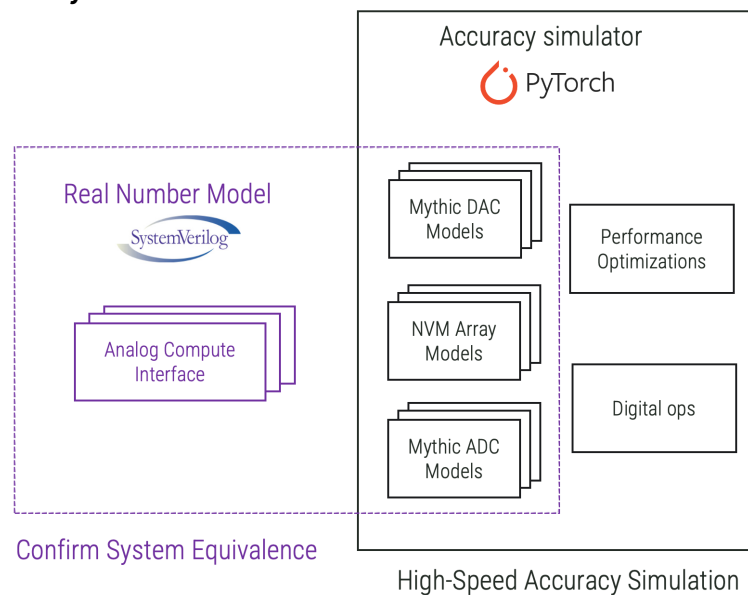
2. SystemVerilog / Real Number Modeling

- Real Number Modeling is a Verilog enhancement that allows digital simulators to model analog behavior with real numbers (and not just binary or logical values).

MYTHIC

- RNMs allow mixed-signal designs to be verified with the same powerful and large-scale methods as Digital Verification
 - Having a SystemVerilog / RNM model of the ACE allows Mythic to verify the ACE the same way all digital processors are verified, and be verified with all of the surrounding digital circuits in the system
 - Mythic's Real Number Models of the analog components are built with the same **reference models created by SPICE simulation**
 - Digital Verification is capable of simulating an entire ACE, and small simulations of our entire chip, but not an entire network
3. PyTorch
- PyTorch is a popular and open-source machine learning framework package in Python
 - PyTorch is how Mythic creates, trains, and analyzes performance of our networks
 - PyTorch is how to simulate an entire network and compare accuracy using the **reference models for computation**

Figure 14. Verilog and Accuracy Simulators Use The Same Reference Models



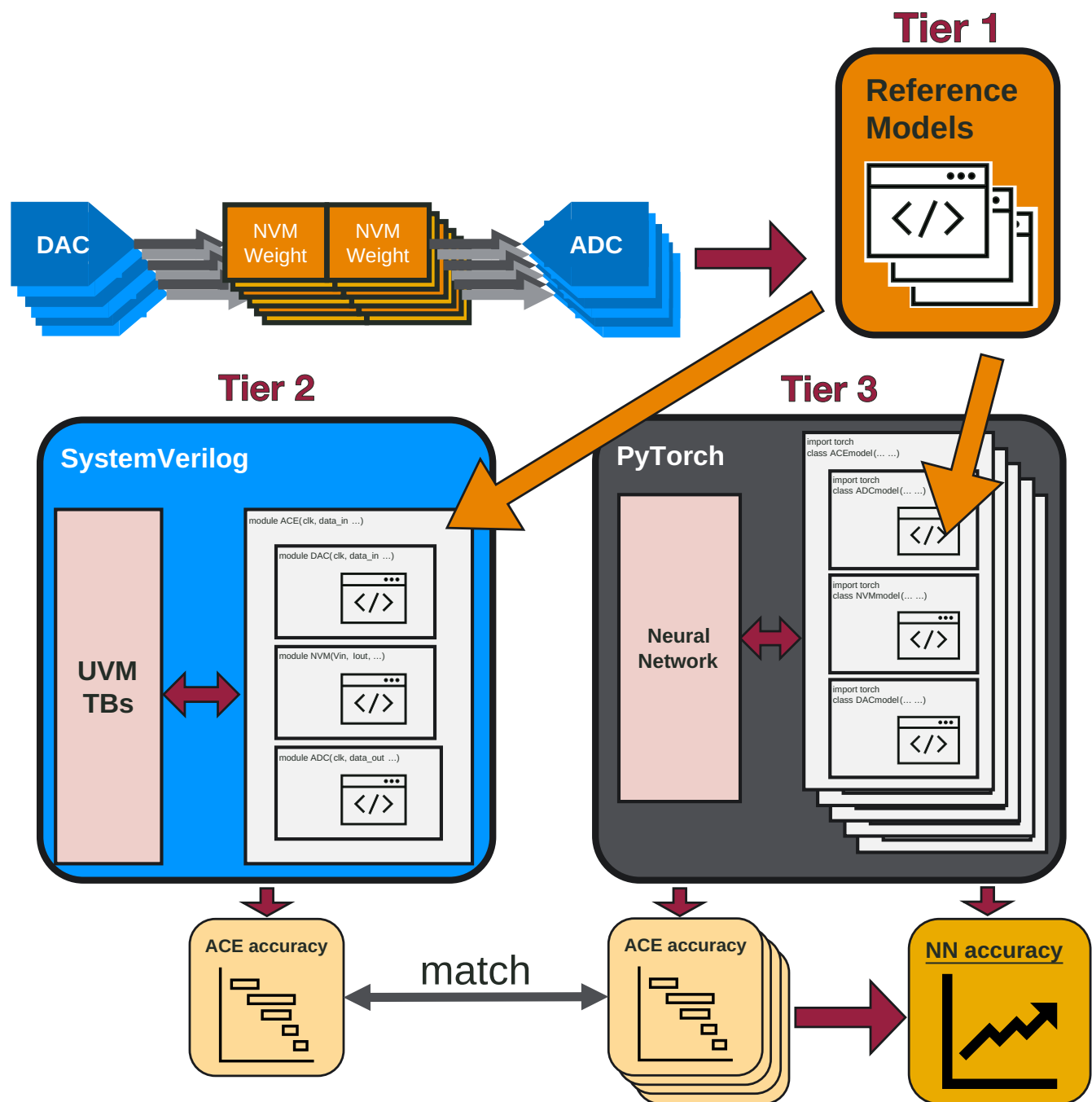
5.1.2 Mythic Modeling Process

The following diagram illustrates the modeling hierarchy described above:

1. First, Mythic circuits team does extensive SPICE simulations to create the **reference models**
2. SystemVerilog Real Number Models are created for the analog circuits that are built with the **same reference models** that allow digital simulation of an entire ACE calculation
3. PyTorch uses the **same reference models** to run an entire network
4. Mythic will confirm computational equivalency between the SystemVerilog computation and the PyTorch computation

MYTHIC

Figure 15. Modeling Process



5.2 Error Sources In Modeling

5.2.1 What Is and Is Not Being Modeled

Mythic is only modeling errors and noise that is expected within integrated circuits. Mythic is not modeling external noise sources such as electromagnetic interference or ion impact.

Examples of Errors Being Modeled

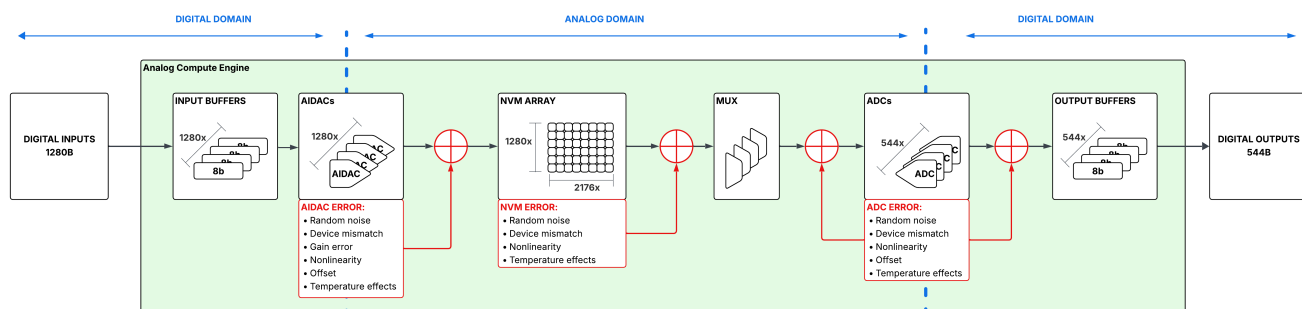
- Manufacturing variation / mismatch - related errors
 - nonlinearity
 - gain error
 - device variation
 - process corners
- Random noise - related errors
 - Thermal noise
 - Flicker noise
- Temperature - related errors
 - device variation
 - noise variation
 - reference variation
 - m40C - 125C
- System-level errors
 - Power supply variation
 - layout / manufacturing parasitics
 - tuning/calibration error

Examples of Errors NOT Being Modeled

- Electromagnetic interference
- Ion impact
- Other environmental noise sources

5.2.2 ACE Signal Chain With Error Sources

The diagram below illustrates how errors from the entire analog signal chain are being modeled and included:



6 ACE Design For Manufacturing

6.1 Built-In-Self-Test

In addition to traditional testing methods, the ACE will contain extensive Built-In-Self-Test (BIST) capabilities. Every AIDAC and ADC will be testable in multiple locations of their components.

Figure 16. AIDAC BIST Diagram

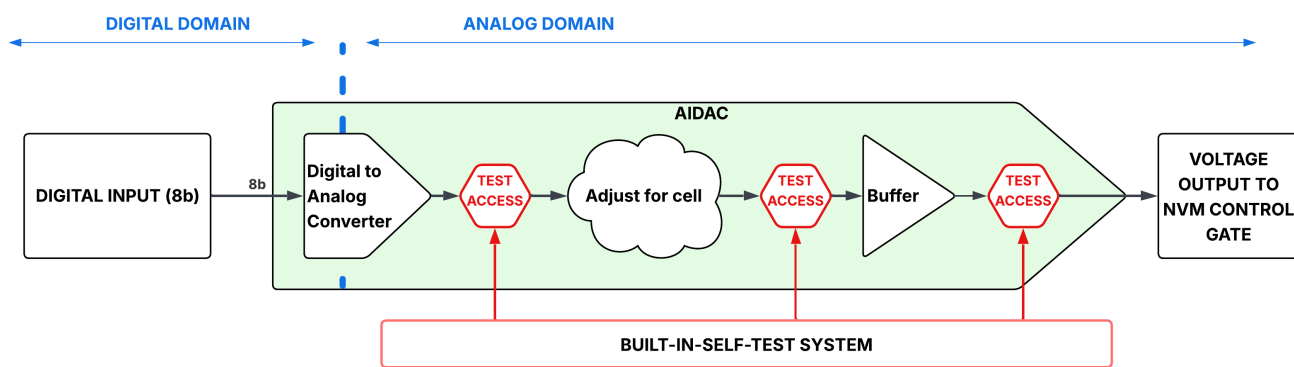
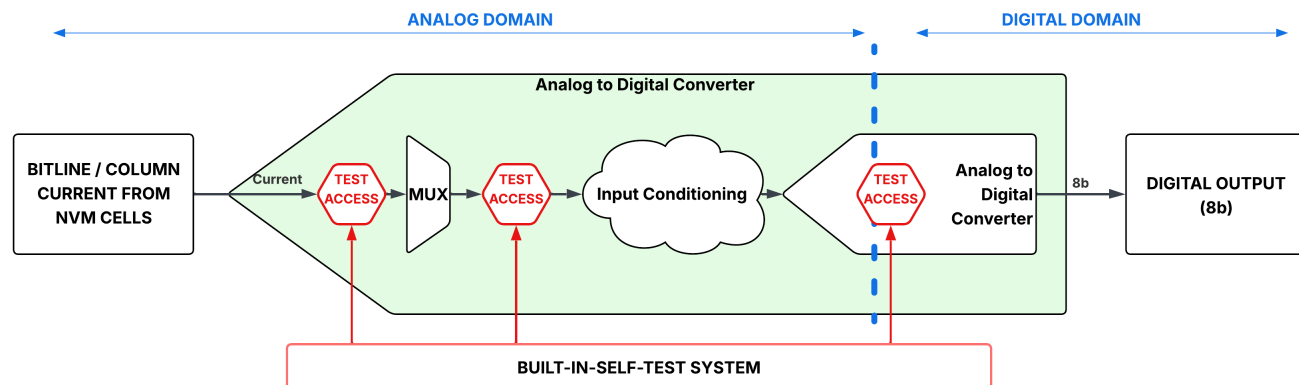


Figure 17. ADC BIST Diagram



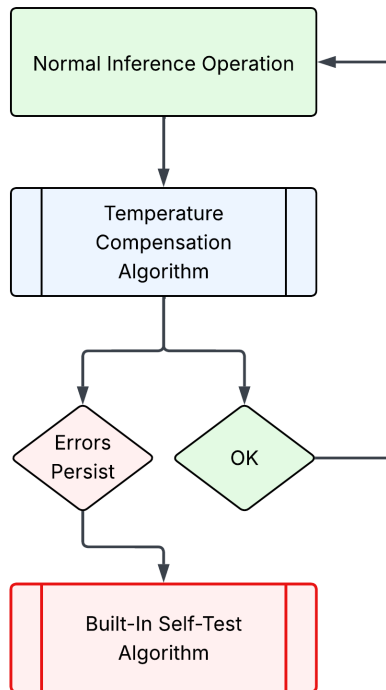
Because every AIDAC and ADC are testable, every row and column of the NVM array will also be testable. All reference circuits are tunable and programmable. The BIST system is being designed such that diagnostics and repair can be run at any time in the field.

6.2 Circuit Monitoring and Temperature Compensation

In order to ensure correct operation across a wide range of temperature, the ACE will regularly run a temperature compensation algorithm which adjusts bias and tuning settings for the circuits to maintain desired operation.

This also functions as a pre-screening for failures, such that if unexpected behavior occurs during routine temperature calibration, the system will be alerted that a full BIST routine should be run.

Figure 18. Monitoring / BIST Flow

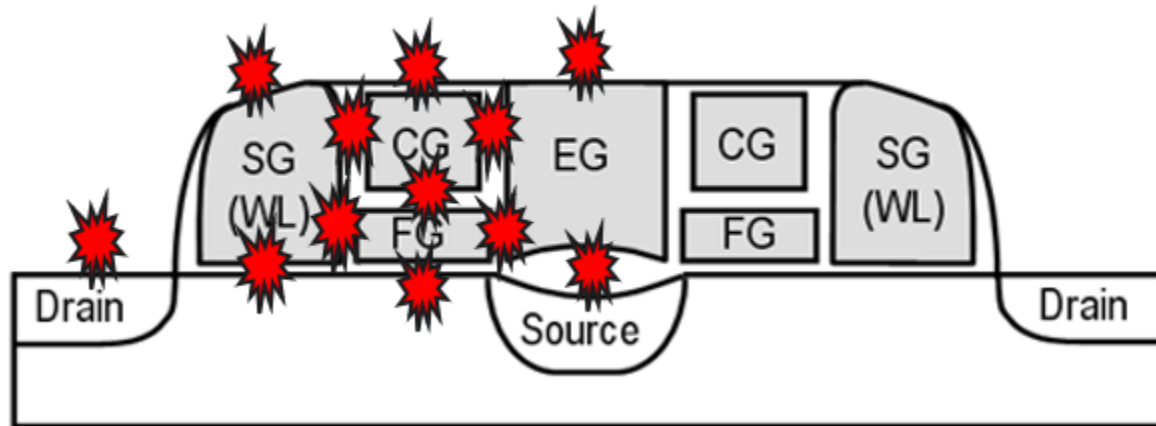


6.3 NVM Cell Failures, Testing and Yield Enhancement:

- Cell Failures
 - Cell failures can be generally categorized in 3 groups: hard defects, soft defects, and outliers
 - Hard Defects are Shorts and Opens from manufacturing defects; They can be anywhere and affecting WL, BL, CG, FG, SL, EG, etc
 - Soft Defects are very weak shorts or weak opens from manufacturing defects; They are manufacturer defects that miss in the beginning of testing (rare but possible). These soft defects could become bad over a period of time
 - Outliers are typically cells which are either too slow or too fast to program and can't achieve the proper weight target
 - Slow cells could cause programming performance issues
 - Fast cells could cause over programming

MYTHIC

Figure 19. Possible locations of defects



- Testing and Detection
 - Time-Zero:
 - NVM arrays will go through multiple sort and test flows at different temperatures to guarantee long-term reliability of the flash cells lifetime and catch all manufacturer defects
 - Tests include stress, DC, AC, screen and functional performance like programming algo
 - Any cell defects include strong/weak short/open and outliers are identified, marked bad (disabled), and information store in efuses to be digitally remapped with good unused cells/ ADCs/AIDACs
 - In-the-field:
 - Programming Algo and BIST flow can be used to catch grown soft defects or new outliers. We can judge if a weight is not changing or changing too fast during programming sequence.

Figure 20. Failures and Detection Methods

Types Failures	Test / Flow Detection	
	Time Zero (At test)	In-the-field
Hard Defects	Stress / AC / DC / Temperature Screen Program Verification BIST	-
Soft Defects	Stress / AC / DC / Temperature Screen Program Verification BIST	Program Verification BIST
Outliers	Program Verification BIST	Program Verification BIST

- Yield:
 - Flash arrays will go through multiple stress test flows at different temperatures to guarantee long-term reliability of the flash cells lifetime
 - Flash yield expected to be very high due to very small ACE array size (~2Mb vs typical 64Mb on 28nm technology node). Average yield after repaired expectation ~ 99.2%
 - Stubborn bits (slow cells) ~ 0.8% to be replaced by redundant cells

6.4 Redundancy

- The array rows and AIDACs, as well as the array columns and ADCs are all identical and completely interchangeable with each other; there is no penalty from replacing a certain row or column with another
- Time-Zero:
 - Any defects include short/open for cells, ADCs and AIDACs are identified, marked bad (disabled), and information store in efuses
 - With this information, we digitally remap with good unused cells/ADCs/AIDACs
- In-Field:
 - Any new defects are identified(disabled) through training/retraining/programming and information will be similarly stored in efuses for remapping with good unused cells/ADCs/AIDACs
 - The Built-In-Self-Test system described above can be run at any time to provide additional diagnostics.